Обзор статьи

Characterization of Large Language Model Development in the Datacenter

Характеризация разработки больших языковых моделей в дата-центре

Студенты АлтГТУ, группа 8 ПИ-51
Бельш А.Е.
Воробьев К.В.
Мэн Ч.
Ошурков Н.А.

2026 г.

Анализ рабочих нагрузок LLM в дата-центре Acme

Крупные языковые модели достигли впечатляющих результатов, но их разработка на больших GPU-кластерах сопряжена с особыми проблемами: частые сбои, сложные стратегии параллелизации и несбалансированное использование ресурсов.
В этой работе представлено шестимесячное эмпирическое исследование трассировок из Acme (кластеры Seren и Kalos, всего 4704 GPU A100) и разработаны системные решения для повышения надежности и эффективности.

Контекст и мотивация

Большие языковые модели (LLM) требуют миллиардов параметров и огромных объёмов данных, поэтому компании создают крупные GPU-кластеры для их обучения.
Однако многие выводы из прежних анализов глубокого обучения (DL) неприменимы к LLM из-за:

- новой парадигмы: масштабное самоконтролируемое предварительное обучение (self-supervised pretraining);
- унифицированной архитектуры: все LLM используют Transformers;
- специализированного программного стека: Deepspeed, Megatron, Alpa и др.

Контекст и мотивация

Большие языковые модели (LLM) требуют миллиардов параметров и огромных объёмов данных, поэтому компании создают крупные GPU-кластеры для их обучения.
Однако многие выводы из прежних анализов глубокого обучения (DL) неприменимы к LLM из-за:

- новой парадигмы: масштабное самоконтролируемое предварительное обучение (self-supervised pretraining);
- унифицированной архитектуры: все LLM используют Transformers;
- специализированного программного стека: Deepspeed, Megatron, Alpa и др.

Методология: трассировки Acme

Анализ основан на данных за 6 месяцев (март–август 2023) из двух кластеров для LLM:

- **Seren:** 2288 GPU A100;
- **Kalos:** 2416 GPU A100.

Источники данных:

- журналы планировщика задач;
- мониторинг инфраструктуры (Prometheus, NVIDIA DCGM);
- журналы выполнения и профилирование.

Трассировки охватывают сотни тысяч заданий CPU/GPU и детальные метрики оборудования.

Ключевые наблюдения — время выполнения и задержки

- **Короткая продолжительность заданий:** Средняя длительность заданий LLM в 2,7–12,8 раз короче, чем в ранее опубликованных DL-трассировках. Это связано с множеством краткосрочных задач, таких как оценка и тюнинг.
- **Несправедливая задержка в очереди:** Короткие оценочные задания испытывают наибольшие задержки в очереди, потому что ресурсы преимущественно резервируются для долгих предобучающих задач (pretraining).

Ключевые наблюдения — использование ресурсов

Несбалансированное потребление:

- Предобучение (pretraining) — 3.2% заданий, но потребляет 94.0% GPU-времени (Kalos);
- Оценка (evaluation) — 92.9% заданий, но использует только 0.8% ресурсов.

Недоиспользование сопутствующих ресурсов:

- CPU, хост-память и сеть часто не загружены;
- GPU и память GPU показывают высокие медианные значения:
  - Kalos: GPU-память медиана 75% (60 ГБ), GPU-использование 99%

Проблемы в оценочных заданиях

Профилирование показало значительное время ожидания и простоя GPU в оценочных конвейерах:

- Пример: HumanEval тратит 29.5% времени на загрузку модели и предобработку данных;
- 19.0% времени уходит на проверку корректности синтезированных программ;
- Лишь около 50% времени используется непосредственно для вывода на GPU.

Следствие: это удлиняет очереди при массовой оценке и увеличивает задержки выполнения заданий.

Системные решения Acme — обзор

На основе наблюдений разработаны две ключевые системы:

1. **Отказоустойчивое предварительное обучение:** для диагностики сбоев и автоматического восстановления.
2. **Раздельное планирование оценки:** для ускорения обратной связи и сокращения простоя GPU.

Отказоустойчивое предварительное обучение

- Частые асинхронные контрольные точки (checkpoints) для быстрого восстановления.
- Диагностика корневых причин с помощью эвристик и LLM-помощника.
- Инструменты для обнаружения неисправных узлов и автоматического перезапуска из корректной контрольной точки.

Эффект: ускорение проверки восстановлений в 3.6–58.7 раза и значительное сокращение ручного вмешательства.

Раздельное планирование оценки и выводы

Развязка извлечения модели:

- Избегает конфликтов при удалённой загрузке модели за счёт отдельной фазы извлечения.

Развязка вычисления метрик:

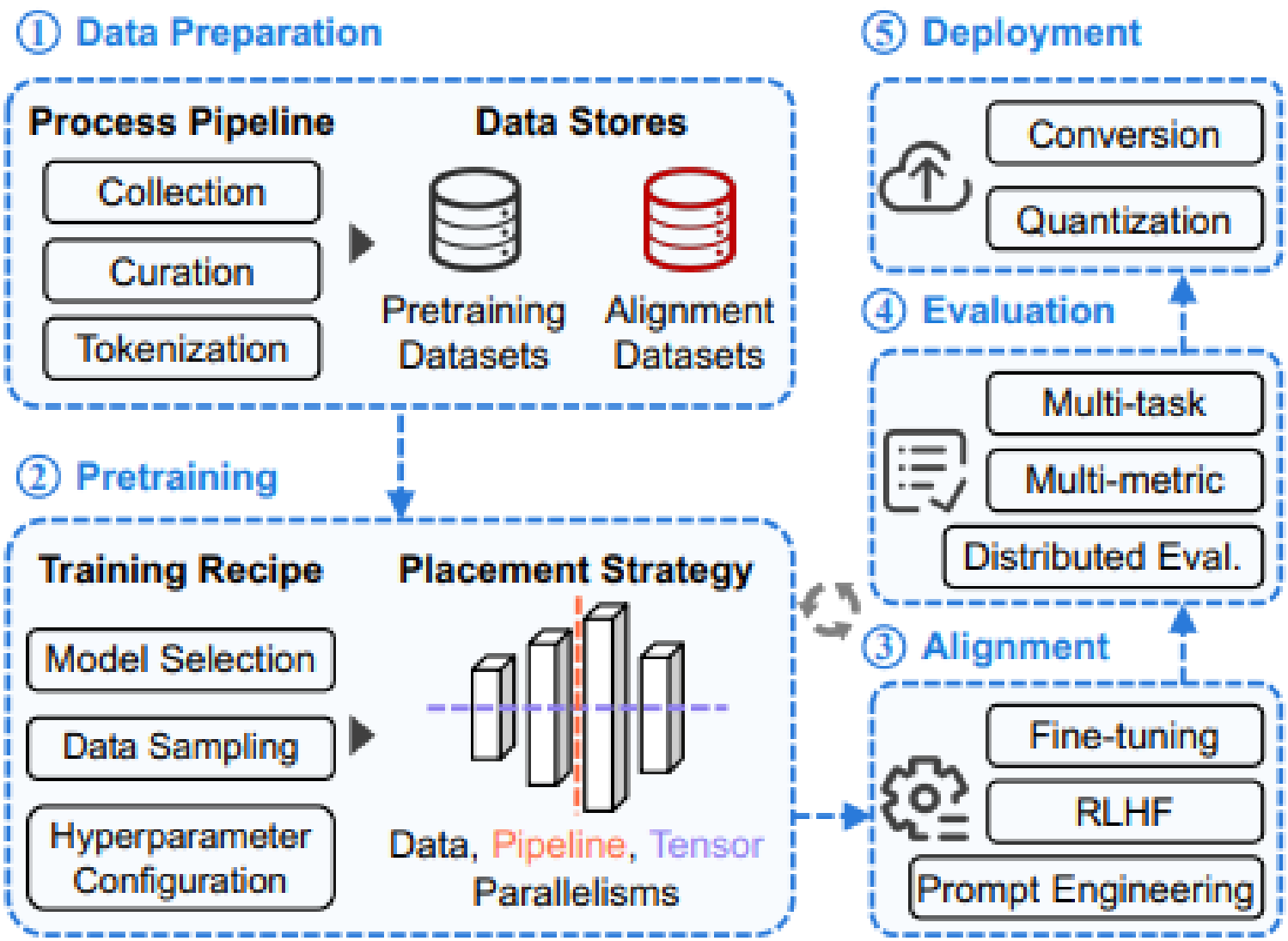
- Минимизирует простой GPU, параллелит вычисление метрик и балансирует нагрузку по предзнаниям и наборам данных.

Практический эффект:

- Сокращение времени оценки до 1.8x.
- Повышена своевременная обратная связь по качеству модели.

Ресурсы: трассировки и код опубликованы на `https://github.com/InternLM/AcmeTrace`.

Конвейер разработки LLM

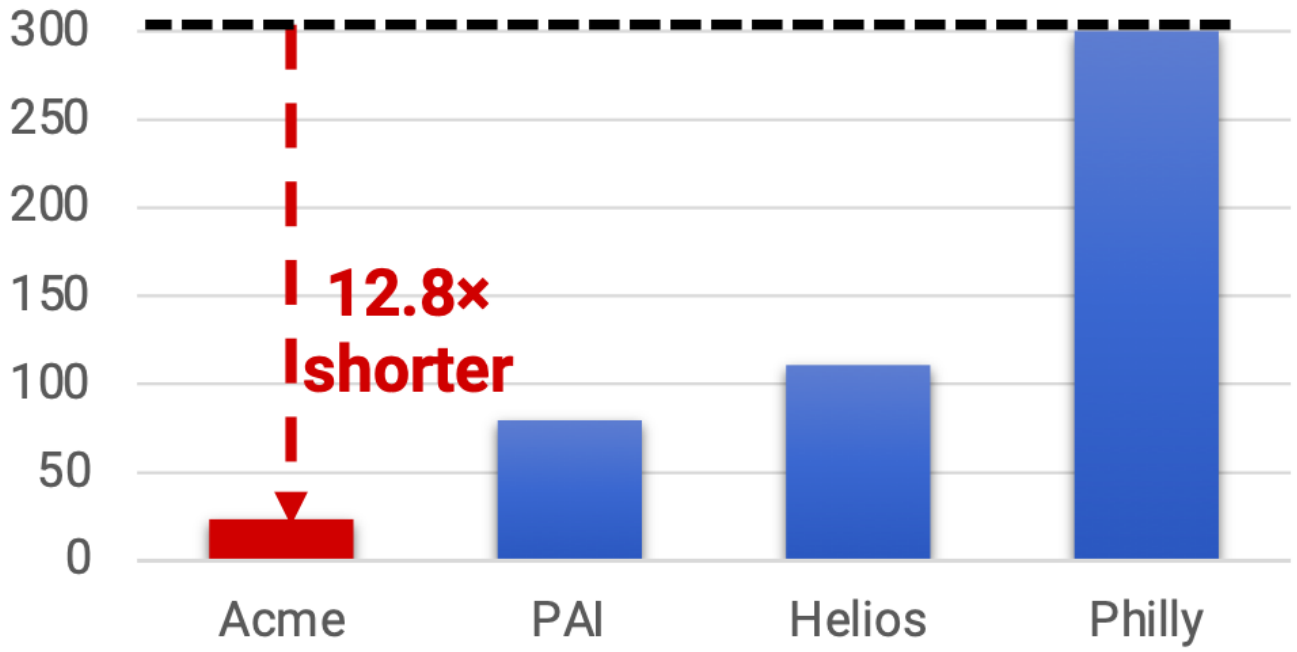


Длительность выполнения задач

Средняя длительность выполнения задач в **2.7–12.8×** *короче*, чем в других DL-трейсах.

Причины:

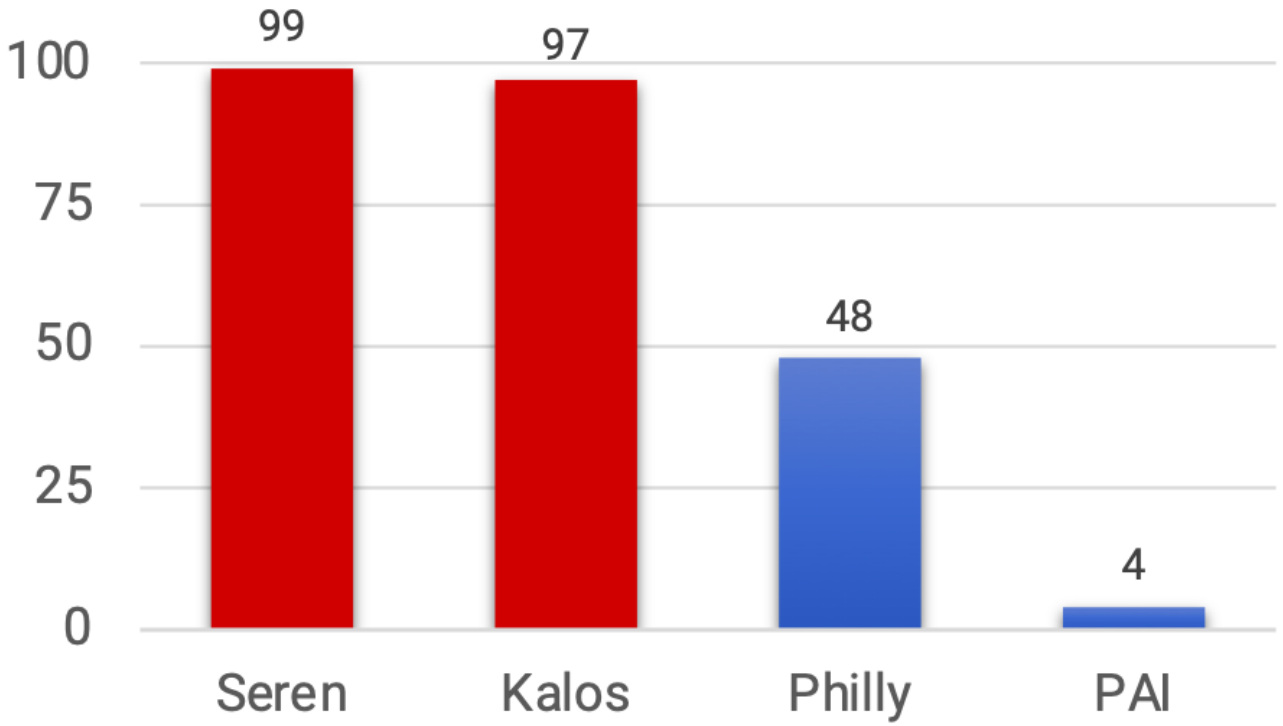
- Много небольших задач для GPU.
- Одна задача использует всего от 5-20 GPU.
- Много задач, которые нельзя выполнить.



среднее выполнение задачи на GPU (минуты)

Нагрузка на GPU

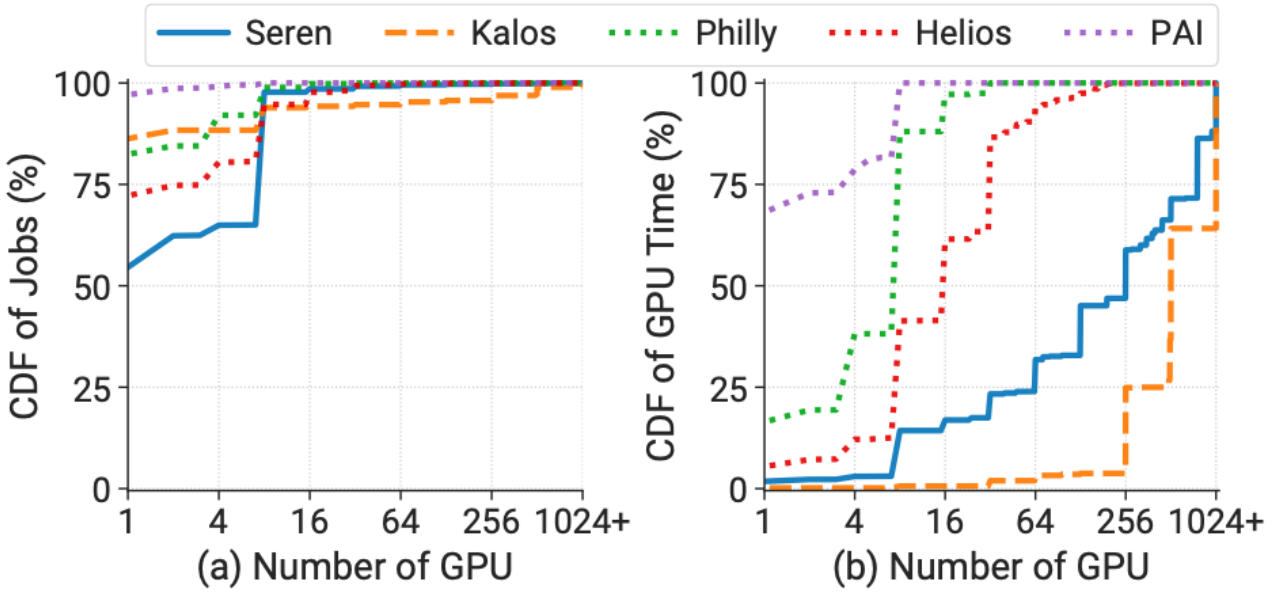
- LLM - вычислительно зависимые системы.
- Много задач не доходит до GPU, отсеиваясь еще на этапе инициализации.



среднее использование GPU

Распределение нагрузки

- Половина задач может выполнятся только на 1 GPU.
- Большую часть времени занимают задачи, где требуется много GPU.



распределение нагрузки в зависимости от количества запрошенных GPU

Задержка в очереди (Queuing Delay)

- **Крупные** задачи претренинга ждут меньше (медиана 28 с).
- **Короткие** задачи оценки — дольше всех (медиана 7.8 мин).
- Причина: приоритетное резервирование ресурсов под претренинг.

figures/queuing_delay.png

Рис. 5: Задержка очереди по типам задач

Потребление GPU-времени (Kalos)

- Претренинг: **3.2%** заданий, но **94%** GPU-времени.
- Оценка: **92.9%** заданий, но лишь **0.8%** GPU-времени.

figures/gpu_time_pie.png

Рис. 6: Распределение GPU-времени

Утилизация GPU: память и вычисления

- Память GPU: медиана **75%** (60 ГБ).
- Вычисления: медиана **99%** (Kalos).
- Значительно выше, чем в старых DL-трейсах (PAI:

figures/gpu_mem_util.png

Рис. 7: CDF использования памяти GPU

Недоиспользование CPU и простои GPU при оценке

- CPU загружен в среднем на **20%** (Kalos).
- В задаче `HumanEval` GPU простаивает **48.5%** времени:
  - 29.5% — загрузка модели / предобработка
  - 19.0% — тестирование программ (CPU)

figures/cpu_util.png

Рис. 9: CDF загрузки CPU

Раздел 4: Дизайн системы и оптимизация

На основе анализа проблем (сбои, задержки оценки) разработаны две системы:

1. **Fault-Tolerant Pretraining (Отказоустойчивый претренинг)**
2. **Decoupled Scheduling for Evaluation (Декомпозированное планирование для оценки)**

Обе интегрированы в фреймворк `InternEvo`.

4.1 Fault-Tolerant Pretraining: Компоненты

- **Асинхронное чекпойнтинг:**
  - Ускоряет сохранение состояния модели в **3.6–58.7x** раз по сравнению с синхронным подходом.
  - Позволяет делать чекпойнты часто, не блокируя обучение.
- **Диагностика с помощью LLM:**
  - Комбинация эвристических правил и LLM для анализа логов.
  - LLM помогает классифицировать и находить первопричины редких или сложных ошибок.
- **Холлистический детектор:**
  - Набор инструментов для точного определения неисправного узла (GPU, сеть, память).
  - Автоматический перезапуск задачи с последнего корректного чекпойнта.
- **Результат:** Снижение необходимости ручного вмешательства и простоев.

4.2 Decoupled Scheduling for Evaluation

- **Проблема:** Длительные простои GPU из-за загрузки модели с удаленного хранилища и CPU-вычислений метрик.

Решения:

- **Декомпозиция загрузки модели:** Кэширование моделей локально на узлах, чтобы избежать повторной загрузки по сети для каждой задачи.
- **Декомпозиция вычисления метрик:** Вынос тестов синтезированных программ (CPU-bound) на отдельные пулы ресурсов, освобождая GPU для инференса.
- **Балансировка на основе знаний о датасетах:** Использование prior knowledge (размер датасета, сложность) для равномерного распределения подзадач по GPU.

Результат: Сокращение общего времени выполнения оценки (*makespan*) до **1.8x**.

beamer [T2A]fontenc [utf8]inputenc [russian]babel []beamerthemeMadrid []beamercolorthemeseahorse

Масштаб исследования

- Дата-центр Acme: 4 704 GPU A100
- 1.09 млн LLM-задач за 6 месяцев
- Среднее число GPU на задачу - 6.3
- Модели от 7B до 123B параметров
- Анализ этапа обучения (serving вынесен в отдельный кластер)

Важно: исследование основано на промышленной эксплуатации, а не на лабораторном эксперименте.

Выявленные системные проблемы

- Падение throughput в 2–3 раза из-за Garbage Collection
- До 30% времени шага занимала функция `list_traverse`
- Утечка памяти в PyTorch Dataloader -> сбой в среднем через 27 часов
- Низкая утилизация GPU в MoE на 1024 GPU
- Узкое место - all-to-all сетевые коммуникации

Вывод: LLM-нагрузки создают новые инфраструктурные риски.